Lycée Duhamel du Monceau. Niveau : Terminale.		
	Numérique et Sciences Informatiques	
	Récursivité	

I) PRESENTATION :

On va dans cette partie, présenter le concept simple mais néanmoins profond de **récursivité**, qui est très proche de la notion mathématique de **récurrence**.

1°) Principe :

Commençons par une définition : un sous–programme (procédure ou fonction) est dit **récursif** s'il s'appelle lui-même.

> Le mot **récursif** provient du latin **recurrere** signifiant courir en arrière. Le lecteur comprendra vite pourquoi ce terme est bien choisi.

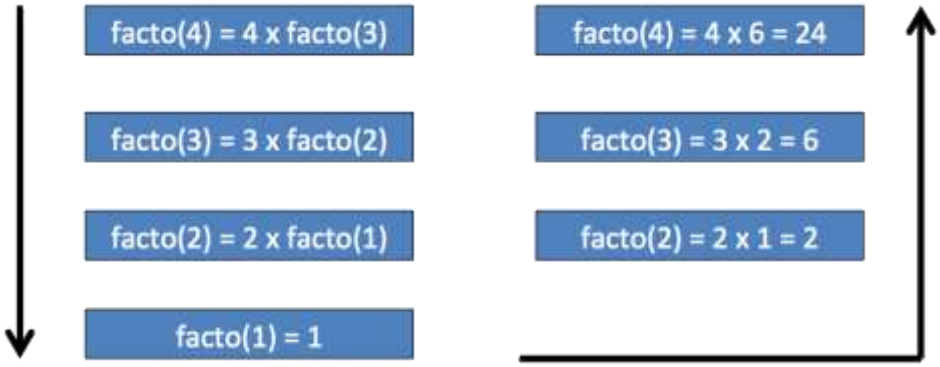
L'idée sous-jacente est que pour résoudre un problème ou effectuer un calcul, on se ramène à la résolution d'un problème similaire mais de **complexité moindre**. On recommence ainsi jusqu'à obtenir un **problème élémentaire** que l'on sait résoudre.
Pour cela, le sous-programme en question va s'appeler lui-même avec un paramètre plus "petit". Cet appel en induira un autre, puis un autre, etc. D'appel en appel, la taille du paramètre va ainsi diminuer. On s'arrêtera quand cette taille sera celle d'un problème immédiatement résolvable.
Les différents problèmes intermédiaires, ceux permettant de passer du problème initial au problème élémentaire, sont stockés successivement en mémoire dans ce que l'on appelle une **pile**. Il s'agit d'une structure de données dans laquelle on accède aux éléments dans l'ordre inverse de leur ajout en mémoire. Dans notre cas, on utilisera ainsi en premier le résultat du problème élémentaire, puis de proche en proche on arrivera à celui du problème initial.

2°) Implémentation récursive de la fonction factorielle :

Rappelons tout d'abord que $n!=1\times2\times3\times...\times n$.
On montre alors facilement la relation de récurrence $n!=n\times(n-1)!$ Si l'on sait calculer $(n-1)!$ on connaîtra donc la valeur de $n!$. Or, toujours d'après la formule de récurrence, $(n-1)!=(n-1)\times(n-2)!$. On est donc ramené au calcul de $(n-2)!$. Et ainsi de suite jusqu'à $1!$ dont on connaît la valeur : 1.
Voici la fonction Python qui utilise cette formule de récurrence pour calculer récursivement la valeur de n! :

```
def factorielleRecursive(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n*factorielleRecursive(n-1)
```

Voyons comment s'exécute cette fonction avec par exemple n=4 :

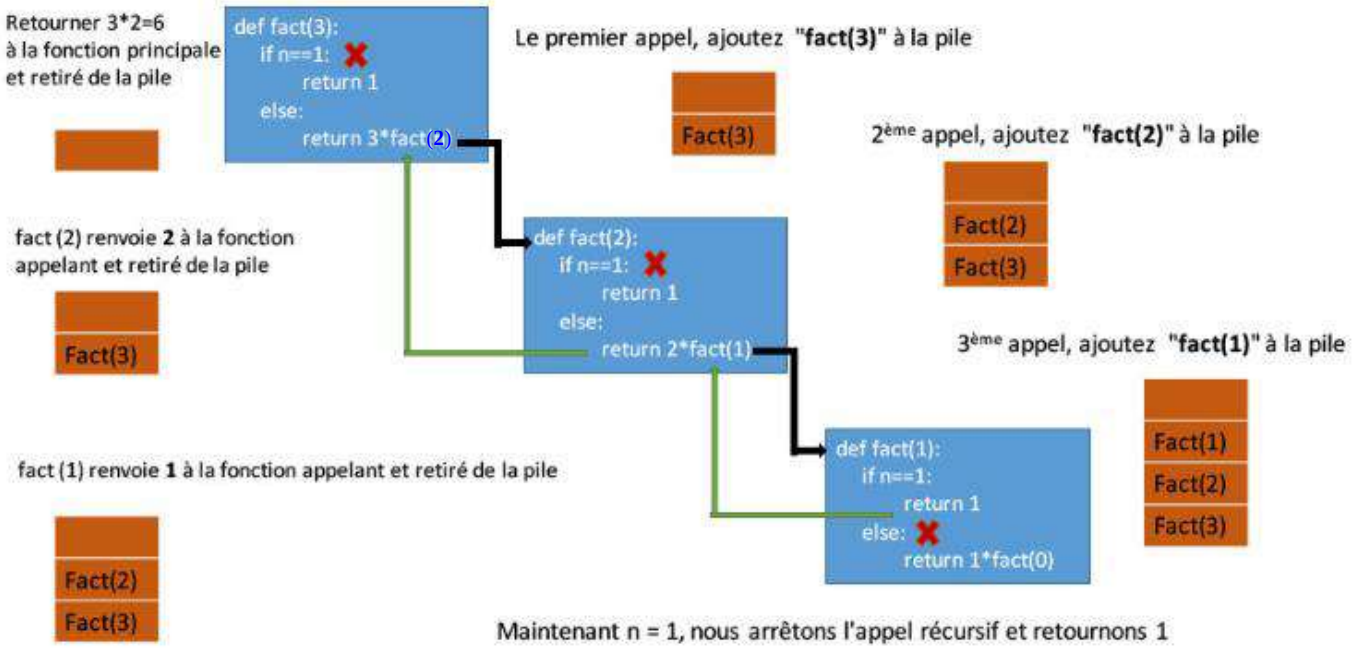


Comme expliqué précédemment, les différents appels récursifs sont empilés en mémoire jusqu'à ce que le paramètre d'appel valle 1 (condition d'arrêt). Ils sont ensuite dépilés jusqu'à l'appel initial.

> Il est indispensable de prévoir une **condition d'arrêt** à la récursion sinon le sous-programme va s'appeler une infinité de fois.

Avec le schéma ci-dessous, c'est peut-être plus clair :

```
def fact(n):
    if n==1:
        return 1
    else:
        return n*fact(n-1)
```



3°) <u>Une erreur à ne pas commettre</u> :

On a omis ici la condition d'arrêt, cette fonction ne se terminera en théorie donc jamais :

```
def factorielleRecursiveBadJob(n):
    return n*factorielleRecursiveBadJob(n-1)
```

En pratique, la pile où sont stockés les appels récursifs étant de taille finie, une fois qu'elle sera pleine le programme ne répondra plus.

4°) <u>Récursivité versus itération</u> :

Par opposition, on qualifiera d'**itératif** un sous-programme qui ne s'appelle pas.
On peut démontrer qu'il est toujours possible de **transformer** un algorithme récursif en un algorithme itératif et inversement.
L'algorithme itératif sera **plus rapide** une fois implémenté dans un langage de programmation mais **souvent plus complexe à écrire**.

<u>Exemple : Implémentation itérative de la fonction factorielle.</u>

Cette fois ci on utilise une structure itérative pour effectuer le calcul :

```
def factorielleIterative(n):
    resultat = 1
    for i in range(2,n+1):
        resultat *= i
    return resultat
```

Il est manifeste que cette version est beaucoup moins élégante que la précédente.

Evoquons à présent quelques arguments en faveur puis en défaveur de l'usage de la récursivité.
Comme nous l'avons déjà mentionné, cette technique de programmation est très **élégante** et **lisible**. Elle évite en effet souvent le recours à de nombreuses structures itératives. Elle est d'autre part très utile voire indispensable pour concevoir des algorithmes sur des structures de données complexes comme les **listes**, les **arbres** et les **graphes**.

L'inconvénient majeur de la récursivité est qu'une fois cette technique implémentée dans un langage de programmation, elle est très "gourmande" en mémoire. Rappelons en effet que l'on doit empiler tous les appels récursifs. Des **débordements de capacité** peuvent donc se produire s'il arrive que cette pile soit pleine.
La taille de la pile de récursion est de 1000 par défaut.
Le lecteur pourra par exemple essayer de calculer 1000! avec la version récursive de la fonction factorielle.

En Python, on peut toutefois ajuster manuellement la taille de la pile dédiée aux appels récursifs grâce à la fonction setrecursionlimit. Mais il y a cependant une taille limite à ne pas dépasser (qui va dépendre aussi de la mémoire du pc).

5°) Récursivité croisée :

Il s'agit d'un cas bien particulier de récursivité où un sous-programme appelle un autre sous-programme qui lui-même appelle le premier.

Exemple : Test de la parité d'un nombre.

Il est clair qu'un nombre n est pair si et seulement si n−1 est impair. De même un nombre n est impair si et seulement si n−1 est pair. De plus 0 est un nombre pair.

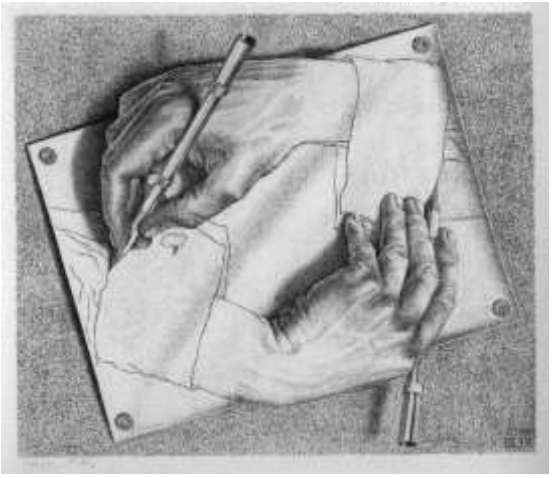
Ces remarques conduisent à cette situation de récursivité croisée :

```
def pair(n):
    if n == 0:
        return True
    else:
        return impair(n-1)

def impair(n):
    if n == 0:
        return False
    else:
        return pair(n-1)
```

Il y a évidemment bien plus simple pour tester la parité d'un nombre... En regardant par exemple si le reste de la division par 2 de ce nombre est égal à 0 ou non.

Figure : "Drawing hands" de Maurits Cornelis Escher.

Une petite minute culturelle, avec cette célèbre lithographie pouvant illustrer le concept de récursivité croisée :



6°) Récursivité multiple :

Lorsqu'un sous-programme récursif réalise plusieurs appels à lui-même, on parle de **récursivité multiple**.

Exemple : Calcul des coefficients binomiaux.

Rappelons tout d'abord la définition d'un coefficient binomial : pour n ∈ N et k ∈ N avec 0 ≤ k ≤ n, on a :

$$\binom{n}{k} = C_n^k = \frac{n!}{k!(n-k)!}$$

On démontre que :

$$\binom{n}{k} = \begin{cases} 1 & \text{si } k = 0 \text{ ou si } k = n \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{sinon} \end{cases}$$

La fonction récursive implémentant cette relation de récurrence réalisera donc deux appels à elle-même :

```
def coeffs(n,k):
    if k == 0 or k == n:
        return 1
    else:
        return coeffs(n-1,k-1) + coeffs(n-1,k)
```

Cette formule de récurrence permet également la construction du très célèbre triangle de Pascal. Voir cours 1SET à ce sujet.

7°) Récursivité imbriquée :

Lorsqu'un sous-programme récursif dont l'appel à lui-même contient un autre appel à lui-même, on parle de **récursivité imbriquée**.

Exemple : La fonction 91 de McCarthy.

Cette fonction est définie sur Z par

$$f(n) = \begin{cases} n - 10 & \text{si } n > 100 \\ f(f(n+11)) & \text{si } n \leq 100 \end{cases}$$

On obtient donc une fonction dont l'appel récursif contient un autre appel récursif :

```
def f91(n):
    if n > 100:
        return n-10
    else:
        return f91(f91(n+11))
```

Cette fonction est certes anecdotique mais son auteur ne l'est pas. John McCarthy (1927-2011) fut en effet l'un des fondateurs de l'intelligence artificielle moderne.

II) REVENONS EN DETAIL SUR LA NOTION DE PILE D'EXECUTION :

À faire vous-même 1

Analysez puis testez le programme suivant :

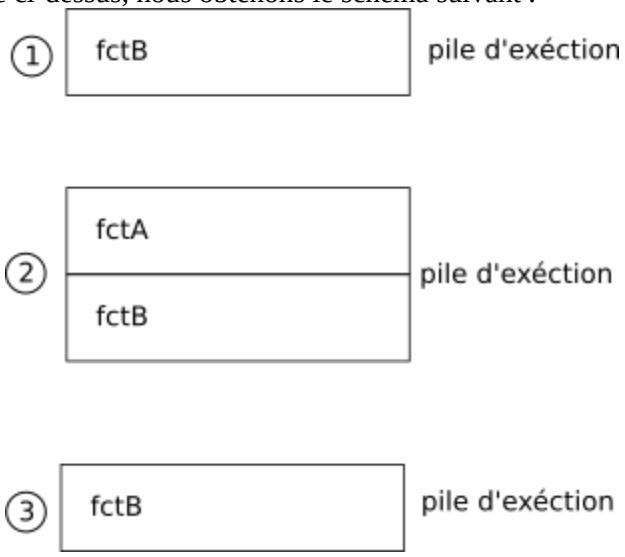
```
def fctA():
    print ("Début fonction fctA")
    i=0
    while i<5:
        print("fctA {}".format(i))
        i = i + 1
    print ("Fin fonction fctA")

def fctB():
    print ("Début fonction fctB")
    i=0
    while i<5:
        if i==3:
            fctA()
            print("Retour à la fonction fctB")
        print("fctB {}".format(i))
        i = i + 1
    print ("Fin fonction fctB")

fctB()
```

Vous devriez obtenir l'enchainement suivant :

```
Début fonction fctB
fctB 0
```

```
fctB 1
fctB 2
Début fonction fctA
fctA 0
fctA 1
fctA 2
fctA 3
fctA 4
Fin fonction fctA
Retour à la fonction fctB
fctB 3
fctB 4
Fin fonction fctB
```

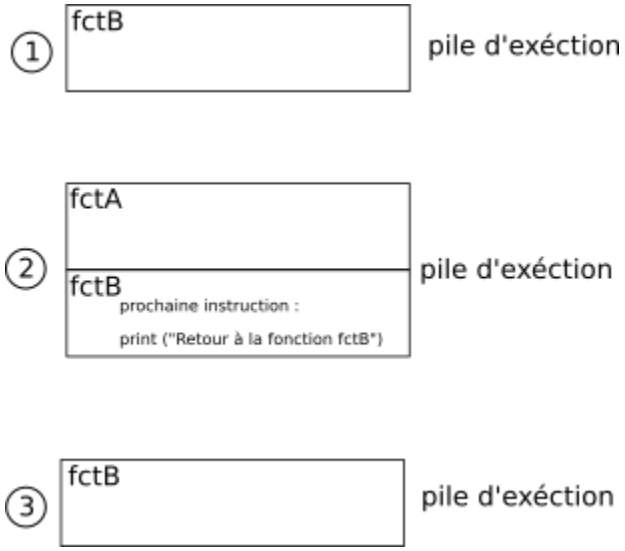
Dans l'exemple ci-dessus, nous avons une fonction (fctB) qui appelle une autre fonction (fctA). La principale chose à retenir de cet exemple est que l'exécution de fctB est interrompue pendant l'exécution de fctA. Une fois l'exécution de fctA terminée, l'exécution de fctB reprendra là où elle avait été interrompue.
Pour gérer ces fonctions qui appellent d'autres fonctions, le système utilise une "pile d'exécution". Une pile d'exécution permet d'enregistrer des informations sur les fonctions en cours d'exécution dans un programme. On parle de pile, car les exécutions successives "s'empilent" les unes sur les autres. Si nous nous intéressons à la pile d'exécution du programme étudié ci-dessus, nous obtenons le schéma suivant :



Nous pouvons "découper" l'exécution de ce programme en 3 parties :

1. la fonction fctB s'exécute jusqu'à l'appel de la fonction fctA
2. l'exécution de la fctB est mise en "pause" pendant l'exécution de la fonction fctA
3. une fois que l'exécution de fctA est terminée, on termine l'exécution de la fonction fctB

Il est important de bien comprendre que la fonction située au sommet de la pile d'exécution est en cours d'exécution. Toutes les fonctions situées "en dessous" sont mises en pause jusqu'au moment où elles se retrouveront au sommet de la pile. Quand une fonction termine son exécution, elle est automatiquement retirée du sommet de la pile (on dit que la fonction est dépilée).
La pile d'exécution permet de retenir la prochaine instruction à exécuter au moment où une fonction sera sortie de son "état de pause" (qu'elle se retrouvera au sommet de la pile d'exécution) :



Évidemment l'explication donnée ci-dessus est quelque peu simpliste : c'est l'adresse mémoire de la prochaine instruction machine à exécuter qui est conservée dans la pile d'exécution
Dans l'exemple ci-dessus, on retrouve une variable i dans les deux fonctions : fctA et fctB. La variable i présente dans la fonction fctA n'a rien à voir avec la variable i présente dans la fonction fctB (elles portent le même nom, mais elles représentent 2 adresses mémoires différentes). Il est très important de bien comprendre que les variables créées dans une fonction ne "sortent" pas de la fonction : chaque fonction possède sa propre liste de variable, comme déjà dit ci-dessus la variable i de la fonction fctB est différente de la variable i de la fonction fctA.
La pile d'exécution conserve une "trace" des valeurs des variables lorsqu'une autre fonction est exécutée. Par exemple la valeur de i (fctB) est conservée au moment de l'exécution de fctA. Quand l'exécution de fctA se termine est que l'exécution de fctB "reprend", la valeur référencée par i (fctB) a été "conservée" (voilà pourquoi on reprend l'exécution de fctB avec un "fctB 3").
Une fonction peut s'appeler elle-même, on parle alors de fonction récursive.

À faire vous-même 2
Analysez puis testez le programme suivant :

```
def fctA():
    print ("Hello")
    fctA()
fctA()
```

Comme vous pouvez le constater, nous avons une erreur dans la console Python :

```
RecursionError: maximum recursion depth exceeded while calling a Python object
```

Dans le cas où une fonction s'appelle elle-même (fonction récursive), on retrouve le même système de pile d'exécution. Dans l'exemple traité ci-dessus, les appels s'enchainent sans rien pour mettre un terme à cet enchainement, la taille de la pile d'exécution augmente sans cesse (aucune fonction ne termine son exécution, nous n'avons pas de "dépilement" juste des "empilements"). Le système interrompt le programme en générant une erreur quand la pile d'exécution dépasse une certaine taille.
Quand on écrit une fonction récursive, il est donc nécessaire de bien penser à mettre en place une structure qui à un moment ou à un autre mettra fin à ces appels récursifs.

Dans le cas de fonctions récursives, il est, comme pour n'importe quelle fonction, possible d'utiliser des paramètres :

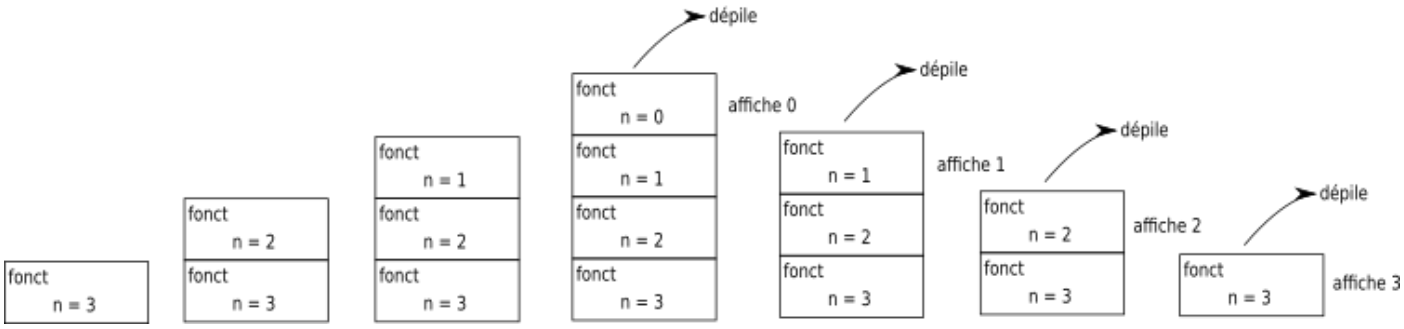
À faire vous-même 3
Essayez de prévoir le résultat de l'exécution du programme ci-dessus. Vérifiez votre hypothèse en exécutant le programme.

```
def fonct(n):
    if n>0:
        fonct(n-1)
    print(n)

fonct(3)
```

Essayons de comprendre en détail ce qui se passe dans le programme ci-dessus :

- 1er appel de la fonction fonct avec le paramètre n = 3 ; n > 0 donc appel de la fonction fonct avec le paramètre n = 2
- 2e appel de la fonction fonct avec le paramètre n = 2 ; n > 0 donc appel de la fonction fonct avec le paramètre n = 1
- 3e appel de la fonction fonct avec le paramètre n = 1 ; n > 0 donc appel de la fonction fonct avec le paramètre n = 0
- 4e appel de la fonction fonct avec le paramètre n = 0 ; n = 0 donc on exécute l'instruction print(n) => affichage : 0
- on "dépile" (3e appel, n = 1) : on exécute l'instruction print(n) => affichage : 1
- on "dépile" (2e appel, n = 2) : on exécute l'instruction print(n) => affichage : 2
- on "dépile" (1er appel, n = 3) : on exécute l'instruction print(n) => affichage : 3

Voici un schéma expliquant le processus en termes de pile d'exécution :



Il ne faut jamais perdre de vue qu'à chaque nouvel appel de la fonction fonct le paramètre n est différent.
Nous allons étudier le calcul de la factorielle grâce à une fonction récursive. D'après Wikipédia : "En mathématiques, la factorielle d'un entier naturel n est le produit des nombres entiers strictement positifs inférieurs ou égaux à n". Par exemple : la factorielle de 3 est : $3 \times 2 \times 1 = 6$; la factorielle de 4 est $4 \times 3 \times 2 \times 1 = 24$; la factorielle de 5 est $5 \times 4 \times 3 \times 2 \times 1 = 120$ ...
Si on note la factorielle de n par n!, on a :

- 0! = 1 (par définition)
- Pour tout entier n > 0, $n! = n \times (n - 1)!$

Nous allons utiliser cette définition de la factorielle pour définir notre fonction récursive (nous allons utiliser le fait que la factorielle de n dépend de la factorielle de n-1 et que 0! = 1) .

À faire vous-même 4

Analysez puis testez la faction fact à l'aide de la console Python :

```
def fact(n) :
   if n > 0 :
      return n*fact(n-1)
   else :
      return 1

print(fact(5))
```

Comme vous pouvez le constater, la fonction fact est structurée de la même manière que la définition mathématique vu ci-dessus :

- dans le cas où n = 0 la fonction renvoie 1 (0! = 1)
- dans le cas où n > 0 la fonction renvoie n*fact(n-1) ($n! = n \times (n - 1)!$)

L'utilisation des fonctions récursives est souvent liée à la notion de récurrence en mathématiques :
En mathématiques une suite définie par récurrence est une suite définie par son premier terme et par une relation de récurrence, qui définit chaque terme à partir du précédent ou des précédents lorsqu'ils existent.
Prenons l'exemple de la suite de Fibonacci qui est définie par :

- $u_0 = 0$, $u_1 = 1$
- et par la relation de récurrence suivante avec n entier et n > 1 : $u_n = u_{n-1} + u_{n-2}$

Ce qui nous donne pour les 6 premiers termes de la suite de Fibonacci :

- $u_0 = 0$
- $u_1 = 1$

- $u_2 = u_1 + u_0 = 1 + 0 = 1$
- $u_3 = u_2 + u_1 = 1 + 1 = 2$
- $u_4 = u_3 + u_2 = 2 + 1 = 3$
- $u_5 = u_4 + u_3 = 3 + 2 = 5$

À faire vous-même 5
En vous aidant de ce qui a été fait pour la fonction fact (voir le "À faire vous-même 4"), écrivez une fonction récursive fib qui donnera le n ième terme de la suite de Fibonacci. Cette fonction prendra en paramètre l'entier n.

III) EXERCICES :

Exercice 1 :
Expliquez les fonctionnalités des fonctions suivantes.

a)
```
def fct1(x,y):
    if x==0:
        return y
    else:
        return fct1(x-1,x+y)
```
b)
```
def fct2(n):
    if n==0:
        return 0
    else:
        return 1+fct2(n//2)
```
c)
```
def fct3(n):
    if n==0:
        return
    fct3(n//2)
    print(n % 2, end=" ")
```
d)
```
def fct1(tab, n):
    if n == 1:
        return tab[0]
    else:
        x = fcttt(tab, n-1)

    if x > tab[n-1]:
        return x
    else:
        return tab[n-1]
```
e)
```
def fct2(a, b):
    if b==0:
        return 1

    if b%2 ==0:
        return fct2(a*a,b//2)

    return fct2(a*a,b//2)*a
```

Exercice 2 :
Étant donné deux entiers positifs n et k, écrire un programme récursif qui affiche toutes les séquences croissantes de longueur k telles que les éléments de chaque séquence est obtenu de n premiers nombres naturels.
Exemple :
k = 2, n = 3
1 2 ; 1 3 ; 2 3

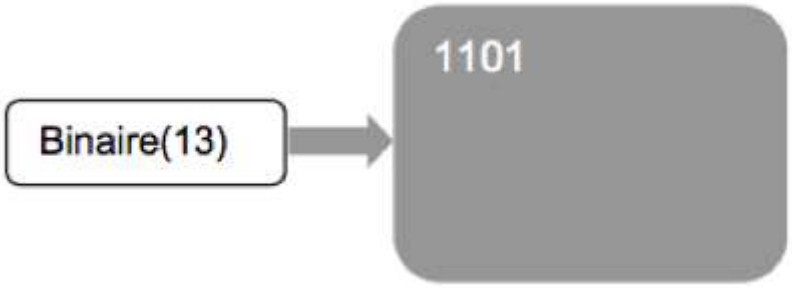
k = 5, n = 5
1 2 3 4 5

Exercice 3 :
Soit une chaine de caractères, écrire un programme récursif permettant de déterminer sa longueur.

Exercice 4 :
Rendre récursive la fonction somme suivante :

```
def somme(L):
   s=0 :
   for val in L :
      s+=val
   return s
```

Exercice 5 :
Pour convertir un nombre entier positif N de la base décimale à la base binaire, il faut opérer par des divisions successives du nombre N par 2. Les restes des divisions constituent la représentation binaire.



Ecrire une fonction récursive « Binaire » permettant d'imprimer à l'écran la représentation binaire d'un nombre N.

Exercice 6 :
Un nombre est pair si est impair, et un nombre N est impair si est pair.
Ecrire deux fonctions récursives mutuelles pair(N) et impair(N) permettant de savoir si un nombre N est pair et si un nombre N est impair.

Exercice 7 :
Soit un tableau X de N entiers, écrire une fonction récursive simple permettant de déterminer le maximum du tableau.

Exercice 8 :
Un tableau X est trié par ordre croissant si, écrire un algorithme récursif permettant de vérifier qu'un tableau X est trié ou non.

Exercice 9 :
Un mot est un palindrome si on peut le lire dans les deux sens de gauche à droite et de droite à gauche. Exemple : KAYAK est un palindrome. Ecrire une fonction récursive permettant de vérifier si un mot est palindrome.

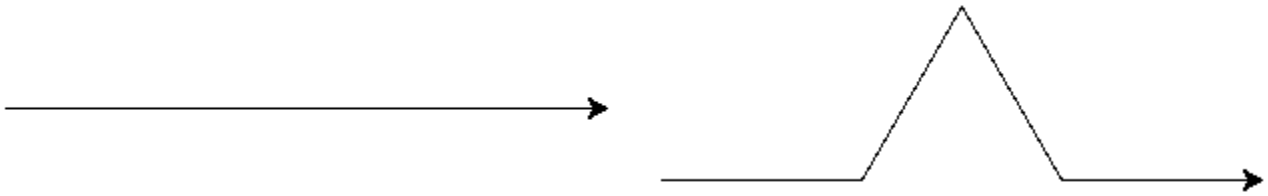
Exercice 10 :
Pour l'exemple suivant, nous allons utiliser le module Python Turtle. Ce module permet dessiner très simplement. Voir Annexe pour plus de détails, ou Étudiez le Wikibook consacré au module Turtle (wikibook Turtle) afin d'acquérir les bases de ce module.

1°) Essayez de prévoir le résultat de l'exécution du programme ci-dessus. Vérifiez votre hypothèse en exécutant le programme.

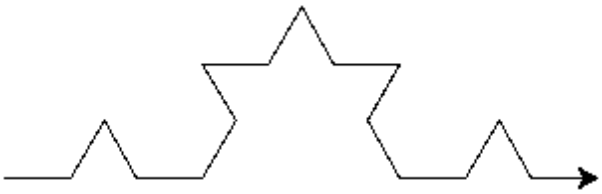
```
# -*- coding: UTF-8 -*-
# programme principal
from turtle import *
forward(100)
left(120)
forward(100)
left(120)
```

```
forward(100)
exitonclick()
```

2°) Flocon de von Koch :
Visionnez la vidéo consacrée au flocon de Koch :
https://youtu.be/PW_Pka9iBko
Le flocon de von Koch s'obtient en partant d'un segment que l'on partage en 3, la partie du milieu est remplacée par un triangle équilatéral comme ceci :



a) Écrire une fonction ligne qui construit cet élément.
b) Écrire une fonction triangle qui construit cet élément (équilatéral).
c) Remplacer les forward de la fonction triangle par un tracé de triangle comme défini précédemment, vous devriez obtenir :



d) Étudier le code suivant :

```
# -*- coding: UTF-8 -*-
# programme principal
from turtle import *
def segment (l,n):
    if n==0:
    # tracer ligne
        forward (l)
    else:
    # dessiner un triangle équilatéral de longueur l/3
        segment (l/3,n-1)
        left (60)
        segment (l/3,n-1)
        right (120)
        segment (l/3,n-1)
        left (60)
        segment (l/3,n-1)

def flocon (l,n):
    segment (l,n)
    right(120)
    segment (l,n)
    right(120)
    segment (l,n)

etape=int(input("Donnez le nombre d'étapes (n) : "))
taille=float(input("Donnez la longueur du côté initial (l) : "))
flocon(taille,etape)
exitonclick()
```

Exercice 11 :
En utilisant le module Turtle, écrire une fonction récursive qui dessine :

ANNEXE : LE MODULE TURTLE

1°) Turtle :
Turtle est une librairie de Python qui permet de réaliser des dessins rudimentaires. Pour utiliser Turtle il faut installer Tkinter qui est souvent compris dans l'installation de Python.
Pour pouvoir utiliser Turtle, il faut tout d'abord, en haut de votre feuille de code importer la librairie turtle :

```
from turtle import *
```

2°) Commandes de base :

```
up() # lève le crayon (pas de dessin).
down() # baisse le crayon.
forward(n) # avance de n.
left(r) # tourne à gauche de r degrés.
right(r) # tourne à droite de r degrés.
goto(x,y) # positionne le crayon en (x,y), (0,0) est le centre de l'écran.
circle(r) # dessine un cercle de rayon r
write('texte') # écrit le texte là où se trouve le curseur
reset() # efface le dessin
width(n) # définit la largeur du trait.
color('blue') # définit la couleur du dessin, ici bleu.
bgcolor('red') # définit la couleur du fond.
```

• Exemple :

```
from turtle import *
color('blue')
forward(150)
left(90)
forward(75)
done()
```

La syntaxe Python pour la définition d'une fonction est assez simple : Il suffit de poser le mot-clef def, suivi du nom de la fonction. Attention à l'indentation du bloc d'instructions !

```
def nomDeLaFonction(liste de parametres):
        ...
        bloc d'instructions
        ...
```

3°) Utilisation :
• Tracer un trait horizontal

```
from turtle import *
def Trait(x,y,n,couleur):
        """(x,y) sont les coordonnées du point de départ.
        n est la longueur du trait"""
        up()
        goto(x,y)
        down()
        color(couleur)
```

```
        forward(n)

Trait(0,0,100,'blue')
done()
```

• Dessiner un carré

```
from turtle import *
def Carre(x,y,n,couleur):
        """(x,y) sont les coordonnées du coin supérieur gauche.
        n est la taille d'un coté"""
        up()
        goto(x,y)
        down()
        color(couleur)
        for i in range(4) :
            forward(n)
            left(90)

Carre(0,0,100,'blue')
done()
```

• Dessiner un polygone régulier

```
from turtle import *
def Poly(x,y,n,c,couleur):
        """(x,y) sont les coordonnées du coin supérieur gauche.
        n est la taille d'un coté
        c est le nombre de cotés"""
        up()
        goto(x,y)
        down()
        angle=360/c
        color(couleur)
        for i in range(c) :
            forward(n)
            right(angle)

Poly(0,0,100,5,'blue')
done()
```

• Dessiner un cercle

```
def Cercle(x,y,r,couleur) :
        """(x,y) sont les coordonnées du centre du cercle
        r est son rayon"""
        Poly(x,y+r/2,r/2,360,'blue')

Cercle(0,0,5,'blue')
done()
```

4°) <u>Donner un petit nom aux tortues</u> :

```
from turtle import * # allows us to use the turtles library
alex = Turtle() # create a turtle named alex
alex.color('blue')
alex.forward(150) # tell alex to move forward by 150 units
alex.left(90) # turn by 90 degrees
alex.forward(75) # complete the second side of a rectangle

tom = Turtle()
tom.right(90)
```

```
tom.color('red')
tom.forward(150)
tom.left(90)
tom.forward(75)

alex.left(90)
alex.forward(150)

tom.left(90)
tom.forward(150)

done()
```

5°) <u>The window background</u> :

Un peu d'anglais ne fait pas de mal . . .
We then create and open what the turtle module calls a screen (we would prefer to call it a window, or in the case of this web version of Python simply a canvas), which we assign to variable wn.
Every window contains a canvas, which is the area inside the window on which we can draw.

```
from turtle import *
wn = Screen()
wn.bgcolor("lightgreen") # set the window background color

tess = Turtle()
tess.color("blue") # make tess blue
tess.pensize(3) # set the width of her pen

tess.forward(50)
tess.left(120)
tess.forward(50)

wn.exitonclick() # wait for a user click on the canvas
```

The wn variable refers to the window. When we invoke its exitonclick method, the program pauses execution and waits for the user to click the mouse somewhere in the window.
When this click event occurs, the response is to close the turtle window and exit (stop execution of) the Python program.

<u>Pour tout savoir sur Turtle</u>:

https://fr.wikibooks.org/wiki/Programmation_Python/Turtle
https://deptinfo-ensip.univ-poitiers.fr/ENS/doku/doku.php/stu:python:turtle
https://mathsp.tuxfamily.org/spip.php?rubrique78
https://docs.python.org/2/library/turtle.html